OOP & C++ Complete Cheatsheet

Based on CS-2002 Course Outline | FAST-NUCES | For Qbatch & WAMO Interviews

1. POINTERS IN C++

Q1. What is a pointer? Declaration and initialization.

A: A pointer stores the memory address of another variable. Declared with *, initialized with & (address-of operator).

```
int x = 10;
int *ptr = &x;   // ptr stores address of x
cout << *ptr;    // dereference -> 10 (value at address)
cout << ptr;     // prints address itself
cout << &x;      // same address as ptr

// Null pointer
int *p = nullptr; // points to nothing
```

Q2. Pointer Arithmetic.

A: Adding/subtracting from a pointer moves it by sizeof(type) bytes, not 1 byte. ptr+1 moves to next element of that type.

```
int arr[5] = {10,20,30,40,50};
int *p = arr;       // points to arr[0]
cout << *p;          // 10
cout << *(p+1);      // 20 (moves 4 bytes for int)
cout << *(p+2);      // 30
p++;                 // now points to arr[1]
cout << *p;          // 20
```

Q3. Pointers and Functions — pass by reference vs pointer.

A: Pass by pointer: function receives address, can modify original. Pass by reference (&): cleaner syntax, same effect.

```
// Pass by pointer
void increment(int *p) { (*p)++; }
int x = 5;
increment(&x;);  // x becomes 6

// Pass by reference
void increment2(int &x;) { x++; }
int y = 5;
increment2(y);  // y becomes 6

// Pass by value (NO effect on original)
void increment3(int x) { x++; } // x is a copy
```

2. DYNAMIC MEMORY ALLOCATION

Q4. Dynamic variables and arrays — new and delete.

A: new allocates memory on heap at runtime. delete frees it. Must always pair new with delete to avoid memory leaks.

```
// Dynamic single variable
int *p = new int;
*p = 10;
delete p;   // free memory

// Dynamic array
int *arr = new int[5];
arr[0] = 100;
delete[] arr;  // free array memory

// Dynamic 2D array
int **matrix = new int*[3];
for (int i = 0; i < 3; i++)
    matrix[i] = new int[3];

// Free 2D array
for (int i = 0; i < 3; i++)
    delete[] matrix[i];
delete[] matrix;
```

Q5. Shallow Copy vs Deep Copy.

A: Shallow copy: copies pointer addresses — both objects point to SAME memory. Deep copy: allocates NEW memory and copies actual data — independent objects.

```
class Array {
  int *data;
  int size;
public:
  Array(int s) { size=s; data=new int[s]; }

  // SHALLOW COPY (default copy constructor) - BAD
  // Array(const Array &a;) { data = a.data; }
  // both objects' data points to SAME memory
  // delete in one destructor crashes the other

  // DEEP COPY (custom copy constructor) - GOOD
  Array(const Array &a;) {
    size = a.size;
    data = new int[size];          // new memory
    for (int i=0;i        data[i] = a.data[i];       // copy values
  }

  ~Array() { delete[] data; }
};
```

3. STRUCTURES IN C++

Q6. Structures — member variables, functions, arrays of structures.

A: struct groups related data. Members public by default (unlike class where default is private). Can contain functions too.

```
struct Student {
  string name;
  int age;
  float gpa;

  // member function
  void display() {
    cout << name << " " << age << " " << gpa << endl;
  }
};

// Array of structures
Student students[3];
students[0].name = "Ali";
students[0].age = 20;
students[0].gpa = 3.5;
students[0].display();

// Struct and pointers
Student *ptr = &students;[0];
ptr->name = "Ahmed";   // arrow operator
(*ptr).age = 21;       // same as above, dereference + dot
```

Q7. Array vs Structure vs Array of Structures.

A: Array: same data type, contiguous memory. Structure: different data types grouped. Array of structures: multiple records of same structure type.

```
// Array - same type
int marks[5] = {80,90,70,60,85};

// Structure - mixed types
struct Employee {
  string name;
  int id;
  float salary;
};

// Array of structures - multiple records
Employee emps[3];
emps[0] = {"Ali", 101, 50000};
emps[1] = {"Sara", 102, 60000};
// each emps[i] has name, id, salary
```

4. RECURSION

Q8. What is recursion? Base case and recursive case.

A: A function that calls itself. Must have a base case (stopping condition) to avoid infinite recursion.

```
int factorial(int n) {
  if (n == 0) return 1;        // base case
  return n * factorial(n-1);   // recursive case
}
// factorial(5) = 5*4*3*2*1 = 120

// Fibonacci recursive (exponential - bad for large n)
int fib(int n) {
  if (n <= 1) return n;
  return fib(n-1) + fib(n-2);
}
// Time: O(2^n) - very slow
// Better: use iterative with O(n)
```

5. CLASSES, OBJECTS & ENCAPSULATION

Q9. Class vs Struct. Accessors and Mutators (Getters/Setters).

A: class: members private by default. struct: members public by default. Accessors (get) return values, mutators (set) change values — implement encapsulation.

```
class BankAccount {
private:
  double balance;  // private - hidden

public:
  // Mutator (setter)
  void setBalance(double b) {
    if (b >= 0) balance = b;  // validation
  }
  // Accessor (getter)
  double getBalance() {
    return balance;
  }
  // Utility function
  void deposit(double amt) {
    balance += amt;
  }
};

BankAccount acc;
acc.setBalance(1000);
// acc.balance = 1000;  // ERROR - private
```

Q10. 3-File structure — separating interface from implementation.

A: Header (.h) declares class. Implementation (.cpp) defines methods. Main (.cpp) uses the class. This separates WHAT (interface) from HOW (implementation).

```
// BankAccount.h (interface/declaration)
class BankAccount {
private:
  double balance;
public:
  BankAccount();
  void deposit(double amt);
  double getBalance();
};

// BankAccount.cpp (implementation)
#include "BankAccount.h"
BankAccount::BankAccount() { balance = 0; }
void BankAccount::deposit(double amt) { balance += amt; }
double BankAccount::getBalance() { return balance; }

// main.cpp (usage)
#include "BankAccount.h"
int main() {
  BankAccount acc;
  acc.deposit(500);
  cout << acc.getBalance();
}
```

Q11. Copy Constructor and Constructor Overloading.

A: Copy constructor creates new object as copy of existing one. Constructor overloading provides multiple ways to initialize an object.

```
class Point {
  int x, y;
public:
  Point() { x=0; y=0; }                    // default constructor
  Point(int a, int b) { x=a; y=b; }        // overloaded constructor
  Point(const Point &p;) {                  // copy constructor
    x = p.x; y = p.y;
  }
  void display() { cout << x << "," << y; }
};

Point p1(5, 10);     // calls overloaded constructor
Point p2 = p1;       // calls copy constructor
Point p3;            // calls default constructor
```

6. INHERITANCE

Q12. Types of Inheritance — Public, Private, Protected.

A: Public: base public/protected stay same in derived. Private: all base members become private in derived. Protected: public/protected become protected.

```
class Base {
public:    int a;
protected: int b;
private:   int c;  // never inherited
};

class PublicDerived : public Base {
  // a is public, b is protected
};

class PrivateDerived : private Base {
  // a is private, b is private
};

class ProtectedDerived : protected Base {
  // a is protected, b is protected
};
```

Q13. Multiple Inheritance and Ambiguity Errors.

A: A class inherits from 2+ base classes. Ambiguity occurs when both bases have a member with the same name — compiler can't decide which one.

```
class A { public: void show() { cout << "A"; } };
class B { public: void show() { cout << "B"; } };

class C : public A, public B {
  // C has TWO show() functions - AMBIGUOUS
};

C obj;
// obj.show();      // ERROR - ambiguous
obj.A::show();       // OK - specify which one
obj.B::show();       // OK
```

Q14. Function Overriding/Redefining.

A: Derived class provides its own implementation of a base class function with same signature. Use 'virtual' in base for runtime polymorphism.

```
class Animal {
public:
  virtual void speak() { cout << "Some sound"; }
};

class Dog : public Animal {
public:
  void speak() override { cout << "Woof"; } // overriding
};

Animal *a = new Dog();
a->speak();  // "Woof" - runtime polymorphism (virtual)
// Without virtual, would print "Some sound"
```

7. COMPOSITION, ASSOCIATION, FRIEND FUNCTIONS

Q15. Composition vs Association vs Aggregation.

A: Composition: strong ownership, part dies with whole (Car has Engine). Aggregation: weak ownership, part can exist independently (Department has Teachers). Association: general relationship between objects.

```
// Composition - Engine cannot exist without Car
class Engine {};
class Car {
  Engine engine; // Engine destroyed when Car destroyed
};

// Aggregation - Teacher can exist without Department
class Teacher {};
class Department {
  vector teachers; // teachers exist independently
};

// Association - two classes use each other, no ownership
class Doctor {};
class Patient {
  Doctor *doctor; // just a reference/relationship
};
```

Q16. Friend Functions and Friend Classes.

A: A friend function/class can access private/protected members of another class even though it's not a member of that class.

```
class Box {
private:
  int width;
public:
  Box(int w) : width(w) {}
  friend void printWidth(Box b); // friend function
  friend class Display;          // friend class
};

void printWidth(Box b) {
  cout << b.width; // can access private member
}

class Display {
public:
  void show(Box b) {
    cout << b.width; // can access private member
  }
};
```

Q17. Static members and functions. Constant members and 'this' pointer.

A: static: shared across all objects of class, one copy exists. const member: cannot be modified after construction. 'this' pointer: points to current object instance.

```
class Counter {
  static int count;  // shared across all objects
  const int id;       // cannot change after init
public:
  Counter() : id(++count) {}  // initialize const in constructor
  static int getCount() { return count; } // static function

  Counter& setId() {
    return *this; // 'this' pointer returns current object
  }
};
int Counter::count = 0; // define static member outside class

Counter c1, c2, c3;
cout << Counter::getCount(); // 3 (shared count)
```

8. OPERATOR OVERLOADING

Q18. Operator Overloading — basic operators with examples.

A: Redefine how operators work for user-defined types (classes). Use 'operator' keyword.

```
class Complex {
  int real, imag;
public:
  Complex(int r=0, int i=0) : real(r), imag(i) {}

  // Overload + operator
  Complex operator+(const Complex &c;) {
    return Complex(real + c.real, imag + c.imag);
  }
  // Overload == operator
  bool operator==(const Complex &c;) {
    return (real==c.real && imag==c.imag);
  }
  void display() { cout << real << "+" << imag << "i"; }
};

Complex c1(2,3), c2(4,5);
Complex c3 = c1 + c2;  // calls operator+
c3.display();          // 6+8i
```

Q19. Operator Overloading using Friend Functions.

A: When the left operand isn't an object of your class (e.g. int + Complex), use a friend function instead of a member function.

```
class Complex {
  int real, imag;
public:
  Complex(int r, int i) : real(r), imag(i) {}

  // Friend function for: int + Complex
  friend Complex operator+(int num, const Complex &c;) {
    return Complex(num + c.real, c.imag);
  }
  void display() { cout << real << "+" << imag << "i"; }
};

Complex c1(2,3);
Complex c2 = 10 + c1;  // 10 is int, c1 is Complex
c2.display();          // 12+3i
```

9. POLYMORPHISM

Q20. Virtual Functions and Runtime Polymorphism.

A: Virtual function allows derived class to override base class method, and the CORRECT version is called at RUNTIME based on object type, not pointer type.

```
class Shape {
public:
  virtual double area() { return 0; }  // virtual
};
class Circle : public Shape {
  double r;
public:
  Circle(double r) : r(r) {}
  double area() override { return 3.14*r*r; }
};
class Square : public Shape {
  double s;
public:
  Square(double s) : s(s) {}
  double area() override { return s*s; }
};

Shape *shapes[2];
shapes[0] = new Circle(5);
shapes[1] = new Square(4);
for (int i=0;i<2;i++)
  cout << shapes[i]->area(); // calls correct version at RUNTIME
```

Q21. Abstract Classes and Pure Virtual Functions.

A: Pure virtual function (=0) has no implementation in base class — forces derived classes to implement it. A class with at least one pure virtual function is abstract — cannot be instantiated.

```
class Shape {
public:
  virtual double area() = 0;  // pure virtual - no body
  virtual void display() = 0;
};

// Shape s;  // ERROR - cannot instantiate abstract class

class Circle : public Shape {
  double r;
public:
  Circle(double r) : r(r) {}
  double area() override { return 3.14*r*r; }     // MUST implement
  void display() override { cout << "Circle"; }   // MUST implement
};

Circle c(5);  // OK - all pure virtuals implemented
```

10. C++ TEMPLATES

Q22. Function Templates and Class Templates.

A: Templates write generic code that works with any data type. Compiler generates specific version when used.

```
// Function template
template
T maxVal(T a, T b) {
  return (a > b) ? a : b;
}
cout << maxVal(5, 10);       // works with int
cout << maxVal(3.5, 2.1);    // works with double

// Class template
template
class Box {
  T value;
public:
  Box(T v) : value(v) {}
  T getValue() { return value; }
};
Box b1(100);
Box b2("Hello");
```

11. EXCEPTION HANDLING

Q23. Try, Catch, Throw — built-in and custom exceptions.

A: try block contains risky code, throw raises an exception, catch handles it. Custom exceptions extend std::exception.

```
#include

// Custom exception
class InsufficientFunds : public exception {
public:
  const char* what() const throw() {
    return "Insufficient balance!";
  }
};

void withdraw(double balance, double amount) {
  if (amount > balance)
    throw InsufficientFunds();
  balance -= amount;
}

int main() {
  try {
    withdraw(100, 200);
  } catch (InsufficientFunds &e;) {
    cout << e.what();  // "Insufficient balance!"
  } catch (exception &e;) {
    cout << "General error: " << e.what();
  }
}
```

12. STL QUICK REFERENCE

Container	Use Case	Key Operations
vector	Dynamic array	push_back, pop_back, [], size()
stack	LIFO	push, pop, top, empty
queue	FIFO	push, pop, front, back
map	Key-value sorted	insert, find, [], erase
unordered_map	Key-value O(1)	insert, find, [], erase
set	Unique sorted values	insert, find, erase

pair	Two values together	first, second, make_pair

FINAL REVISION — ALL TOPICS IN ONE TABLE

Topic	One-Line Definition
Pointer	Variable storing memory address of another variable
Pointer Arithmetic	ptr+1 moves by sizeof(type) bytes
Dynamic Memory	new allocates heap memory, delete frees it
Shallow Copy	Copies pointer/address - shares same memory
Deep Copy	Allocates new memory, copies actual data
Structure	Groups different data types, public by default
Recursion	Function calling itself with a base case
Encapsulation	Hiding data with private + getters/setters
Abstraction	Hiding implementation, exposing only interface
Inheritance	Child class extends parent class
Multiple Inheritance	Class inherits from 2+ base classes
Ambiguity Error	Same member name in multiple base classes
Function Overriding	Child redefines parent method (virtual)
Function Overloading	Same name, different parameters, same class
Composition	Strong ownership - part dies with whole
Aggregation	Weak ownership - part exists independently
Friend Function	Non-member function with access to privates
Static Member	Shared across all objects, one copy
this Pointer	Points to the current object instance
Operator Overloading	Redefining operators for custom types
Virtual Function	Enables runtime polymorphism via overriding
Pure Virtual Function	No body, =0, makes class abstract
Abstract Class	Has at least 1 pure virtual function
Template	Generic code for any data type
Exception Handling	try-catch-throw for error handling